

**Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки**

Лабораторна робота №1

з дисципліни
«Об'єктно-орієнтоване програмування»

на тему

“Знайомство із середовищем розробки та складання модульних проєктів програм “

Виконав:

студент групи ІМ-о51
Тараєв Олексій Богданович
Номер в списку - 14

Перевірив:

Рекечинський Д.О.

Київ 2026

Завдання:

1. Створити у обраному середовищі розробки проєкт з ім'ям Lab1.
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати вихідний текст і отримати виконуваний файл програми.
4. Перевірити роботу програми. За потреби налагодити програму.
5. Проаналізувати та прокоментувати результати та вихідний текст програми.

Завдання згідно варіанту (Варіант 14)

Розрахунок варіанта:

Номер за списком у журналі: $J = 14$.

Варіант для пункту меню Работа 1: $B1 = 14 \bmod 4 = 2$.

Варіант для пункту меню Работа 2: $B2 = (14 + 1) \bmod 4 = 3$.

Зміст завдань:

Робота1- Варіант 2 Реалізувати двоетапний майстер із двох послідовних діалогових вікон. Перше вікно містить кнопки [Next >] і [Cancel]. Кнопка [Next >] закриває перше вікно та відкриває друге з кнопками [< Back], [OK] та [Cancel]. Кнопка [< Back] повертає користувача до першого вікна, а [OK] фіксує успішне завершення майстра на полотні головного вікна.

Робота 2 - Варіант 3 Реалізувати діалогове вікно з елементом списку ListBox та кнопками [OK] і [Cancel]. У список вносяться назви академічних груп потоку (IM-51, IM-52, IM-53, IM-54, IM-o51). Обрана група виводиться по центру головного вікна додатку.

[module1.py](#)

```
import logging

import tkinter as tk

from tkinter import ttk

__all__ = ["func_mod1"]

logger = logging.getLogger("Lab1.Module1")

class _FirstDialog(tk.Toplevel):

    def __init__(self, parent: tk.Tk) -> None:

        super().__init__(parent)

        logger.info("Initializing FirstDialog window (Step 1).")

        self.title("Step 1 of 2")

        self.geometry("340x160")

        self.resizable(False, False)

        self.transient(parent)

        self.grab_set()

        self._result: int = 0

        self._build_ui()

        self.protocol("WM_DELETE_WINDOW", self._on_cancel)

        self.wait_window(self)
```

```
def _build_ui(self) -> None:

    lbl = ttk.Label(

        self,

        text="Wizard Step 1:\nPress 'Next >' to proceed to the next
step.",

        font=("Segoe UI", 10),

        justify=tk.CENTER,

    )

    lbl.pack(expand=True, padx=20, pady=10)

    btn_frame = ttk.Frame(self)

    btn_frame.pack(side=tk.BOTTOM, pady=15)

    btn_next = ttk.Button(btn_frame, text="Next >",
command=self._on_next)

    btn_next.pack(side=tk.LEFT, padx=10)

    btn_cancel = ttk.Button(btn_frame, text="Cancel",
command=self._on_cancel)

    btn_cancel.pack(side=tk.LEFT, padx=10)

def _on_next(self) -> None:

    logger.info("FirstDialog confirmed: proceeding to Step 2.")

    self._result = 1

    self.destroy()

def _on_cancel(self) -> None:

    logger.info("FirstDialog cancelled by user.")

    self._result = 0

    self.destroy()
```

```
    @property

    def result(self) -> int:

        return self._result


def func_mod1(parent: tk.Tk) -> int:

    logger.info("func_mod1 invoked.")

    dialog = _FirstDialog(parent)

    return dialog.result
```

[module2.py](#)

```
import logging

import tkinter as tk

from tkinter import ttk


__all__ = ["func_mod2"]


logger = logging.getLogger("Lab1.Module2")


class _SecondDialog(tk.Toplevel):

    def __init__(self, parent: tk.Tk) -> None:

        super().__init__(parent)
```

```
logger.info("Initializing SecondDialog window (Step 2).")

self.title("Step 2 of 2")

self.geometry("360x160")

self.resizable(False, False)

self.transient(parent)

self.grab_set()

self._result: int = 0

self._build_ui()

self.protocol("WM_DELETE_WINDOW", self._on_cancel)

self.wait_window(self)

def _build_ui(self) -> None:

    lbl = ttk.Label(

        self,

        text="Wizard Step 2:\nPress 'OK' to finish or '< Back' to\nreturn.",

        font=("Segoe UI", 10),

        justify=tk.CENTER,

    )

    lbl.pack(expand=True, padx=20, pady=10)

    btn_frame = ttk.Frame(self)

    btn_frame.pack(side=tk.BOTTOM, pady=15)

    btn_back = ttk.Button(btn_frame, text("< Back",

command=self._on_back)

    btn_back.pack(side=tk.LEFT, padx=5)
```

```

        btn_ok = ttk.Button(btn_frame, text="OK", command=self._on_ok)
        btn_ok.pack(side=tk.LEFT, padx=5)

        btn_cancel = ttk.Button(btn_frame, text="Cancel",
command=self._on_cancel)

        btn_cancel.pack(side=tk.LEFT, padx=5)

def _on_back(self) -> None:
    logger.info("SecondDialog: navigating back to Step 1.")
    self._result = -1
    self.destroy()

def _on_ok(self) -> None:
    logger.info("SecondDialog confirmed: completing wizard.")
    self._result = 1
    self.destroy()

def _on_cancel(self) -> None:
    logger.info("SecondDialog cancelled by user.")
    self._result = 0
    self.destroy()

@property
def result(self) -> int:
    return self._result

def func_mod2(parent: tk.Tk) -> int:

```

```
logger.info("func_mod2 invoked.")

dialog = _SecondDialog(parent)

return dialog.result
```

module3.py

```
import logging

import tkinter as tk
from tkinter import ttk

__all__ = ["func_mod3"]

logger = logging.getLogger("Lab1.Module3")

class _ListDialog(tk.Toplevel):
    def __init__(self, parent: tk.Tk) -> None:
        super().__init__(parent)

        logger.info("Initializing ListDialog window.")

        self.title("Select Faculty Group")

        self.geometry("320x280")

        self.resizable(False, False)

        self.transient(parent)

        self.grab_set()

        self._result: str | None = None
```

```
self._groups = ["IM-51", "IM-52", "IM-53", "IM-54", "IM-o51"]

self._build_ui()

self.protocol("WM_DELETE_WINDOW", self._on_cancel)

self.wait_window(self)

def _build_ui(self) -> None:

    lbl = ttk.Label(self, text="Select student group from the
list:")

    lbl.pack(anchor=tk.W, padx=20, pady=(15, 5))

    frame = ttk.Frame(self)

    frame.pack(fill=tk.BOTH, expand=True, padx=20, pady=5)

    scrollbar = ttk.Scrollbar(frame, orient=tk.VERTICAL)

    self._listbox = tk.Listbox(

        frame,

        yscrollcommand=scrollbar.set,

        selectmode=tk.SINGLE,

        font=("Segoe UI", 10),

    )

    scrollbar.config(command=self._listbox.yview)

    scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

    self._listbox.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

    for group in self._groups:

        self._listbox.insert(tk.END, group)
```

```

self._listbox.select_set(0)

btn_frame = ttk.Frame(self)
btn_frame.pack(side=tk.BOTTOM, pady=15)

btn_ok = ttk.Button(btn_frame, text="OK", command=self._on_ok)
btn_ok.pack(side=tk.LEFT, padx=10)

btn_cancel = ttk.Button(btn_frame, text="Cancel",
command=self._on_cancel)

btn_cancel.pack(side=tk.LEFT, padx=10)

def _on_ok(self) -> None:
    selection = self._listbox.curselection()

    if selection:
        self._result = self._listbox.get(selection[0])

        logger.info("ListDialog confirmed selection: '%s'",
self._result)

    else:
        self._result = None

        logger.info("ListDialog confirmed with no selection.")

    self.destroy()

def _on_cancel(self) -> None:
    self._result = None

    logger.info("ListDialog cancelled by user.")

    self.destroy()

@property
def result(self) -> str | None:

```

```
        return self._result

def func_mod3(parent: tk.Tk) -> str | None:

    logger.info("func_mod3 invoked.")

    dialog = _ListDialog(parent)

    return dialog.result
```

[main.py](#)

```
import logging

import tkinter as tk

from tkinter import messagebox

from module1 import func_mod1
from module2 import func_mod2
from module3 import func_mod3

logging.basicConfig(

    level=logging.DEBUG,

    format="%(asctime)s [%(levelname)s] (%(name)s): %(message)s",

    datefmt="%H:%M:%S",

)

logger = logging.getLogger("Lab1.Main")

class MainWindow(tk.Tk):

    def __init__(self) -> None:
```

```
super().__init__()

logger.info("Initializing MainWindow.")

self.title("Lab 1: Modular Application (Variant 14)")

self.geometry("640x420")


self._display_text: str = "Select an action from the 'Actions'
menu."


self._init_menu()

self._init_canvas()


def _init_menu(self) -> None:

    logger.debug("Building system menu bar.")

    menubar = tk.Menu(self)

    file_menu = tk.Menu(menubar, tearoff=0)

    file_menu.add_command(label="Exit", command=self.destroy)

    menubar.add_cascade(label="File", menu=file_menu)

    actions_menu = tk.Menu(menubar, tearoff=0)

    actions_menu.add_command(label="Work 1",
command=self._on_work1)

    actions_menu.add_command(label="Work 2",
command=self._on_work2)

    menubar.add_cascade(label="Actions", menu=actions_menu)

    help_menu = tk.Menu(menubar, tearoff=0)

    help_menu.add_command(label="About", command=self._on_about)

    menubar.add_cascade(label="Help", menu=help_menu)
```

```
self.config(menu=menubar)

def _init_canvas(self) -> None:

    logger.debug("Configuring canvas area.")

    self._canvas = tk.Canvas(self, bg="#FFFFFF")

    self._canvas.pack(fill=tk.BOTH, expand=True)

    self._canvas.bind("<Configure>", lambda _event: self._redraw())

def _redraw(self) -> None:

    self._canvas.delete("all")

    width = self._canvas.winfo_width()

    height = self._canvas.winfo_height()

    self._canvas.create_text(

        width // 2,

        height // 2,

        text=self._display_text,

        font=("Segoe UI", 14),

        fill="#111111",

        justify=tk.CENTER,

    )

def _on_work1(self) -> None:

    logger.info("Action dispatched: Work 1 (Sequential Dialogs).")

    step = 1

    while True:

        if step == 1:

            res1 = func_mod1(self)
```

```

        if res1 == 1:

            step = 2

        else:

            logger.info("Work 1 terminated at Step 1.")

            break

    elif step == 2:

        res2 = func_mod2(self)

        if res2 == -1:

            step = 1

        elif res2 == 1:

            logger.info("Work 1 successfully completed.")

            self._display_text = (

                "Result of Work 1:\n\nTwo-step wizard
successfully completed."

            )

            self._redraw()

            break

        else:

            logger.info("Work 1 terminated at Step 2.")

            break

def _on_work2(self) -> None:

    logger.info("Action dispatched: Work 2 (ListBox Selection).")

    group = func_mod3(self)

    if group is not None:

        logger.info("Work 2 completed with group: '%s'.", group)

        self._display_text = f"Result of Work 2 (Selected
Group):\n\n{group}"

        self._redraw()

    else:

```

```
        logger.info("Work 2 dismissed without selection.")

def _on_about(self) -> None:

    logger.info("Help action triggered: About dialog.")

    messagebox.showinfo(

        "About",

        "Laboratory Work #1\nVariant 14 (B1=2, B2=3)\nModular GUI\nArchitecture",

    )

def main() -> None:

    logger.info("Starting main application loop.")

    app = MainWindow()

    app.mainloop()

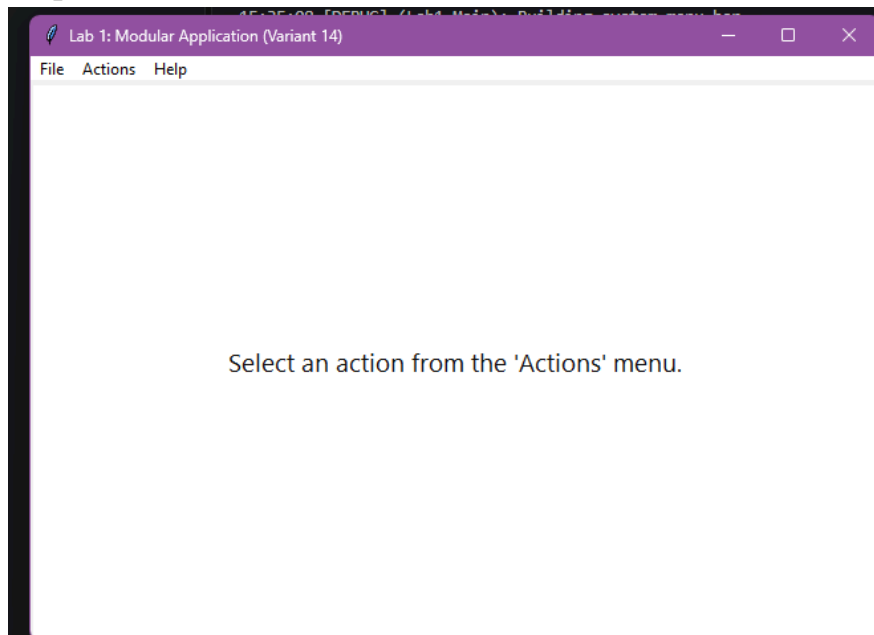
    logger.info("Application loop terminated.")

if __name__ == "__main__":

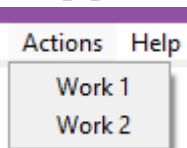
    main()
```

скріншоти

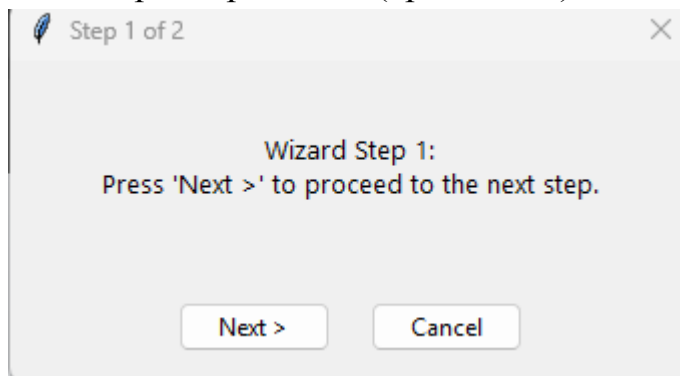
стартове меню

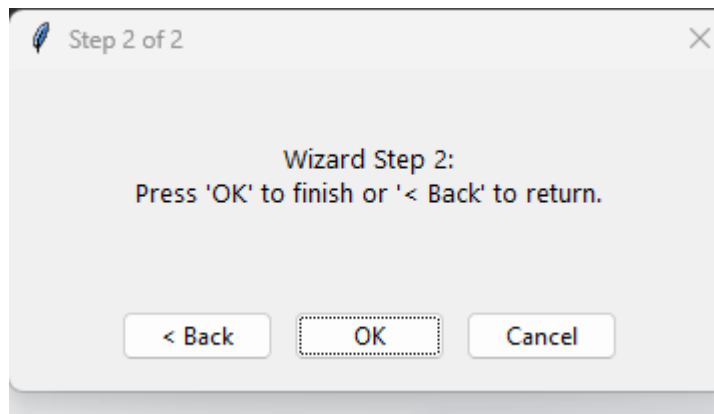


вибір роботи

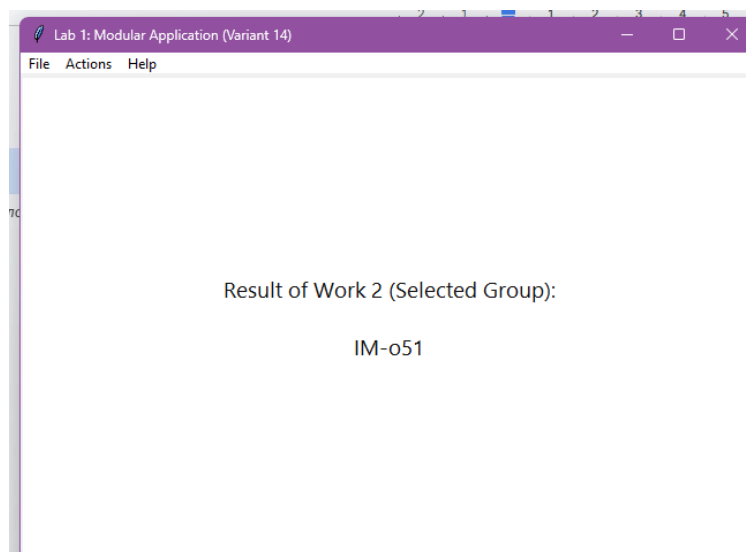
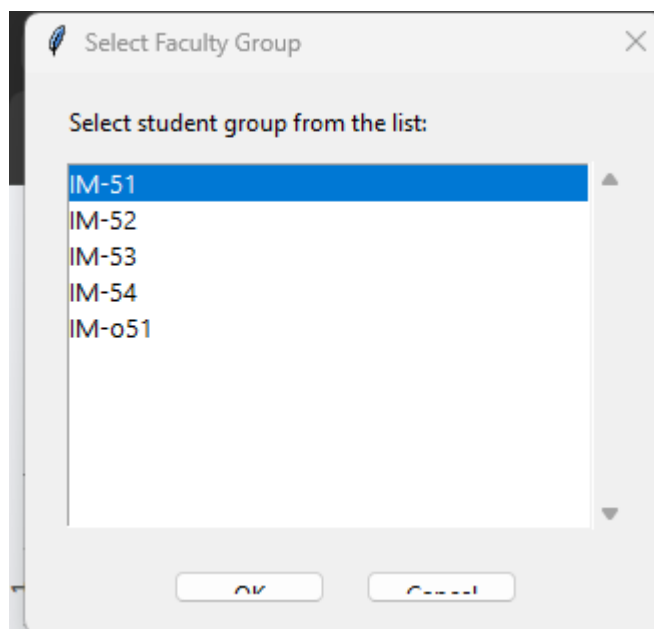


демонстрація роботи 1 (крок 1 та 2)

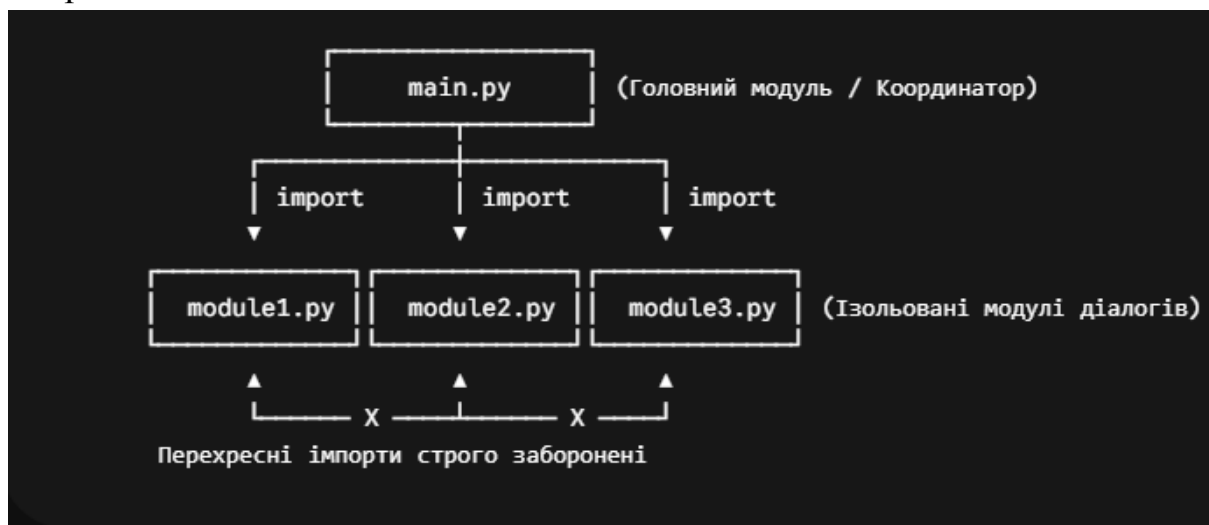




демонстрація роботи 2



діаграма залежностей



Висновки

Під час виконання лабораторної роботи №1 було створено віконний графічний додаток на мові Python із модульною архітектурою.

Було досягнуто повної незалежності модулів: кожне діалогове вікно винесено в окремий файл а взаємодія між компонентами побудована за ієрархічним принципом без виникнення циклічних залежностей та витoku деталей реалізації діалогів у головний файл. Роботу діалогових вікон синхронізовано з головним циклом подій додатка, дотримано принципи інкапсуляції та вимоги щодо автоматичного закриття попередніх вікон при переходах